Comparative Performance Analysis of MILP Solvers for Cryptanalysis

Halil İbrahim Kaplan
TÜBİTAK BİLGEM, Cryptanalysis Laboratory

April 29, 2026

Abstract

This paper presents a comprehensive performance comparison of five state-of-the-art Mixed Integer Linear Programming (MILP) solvers when applied to related-key differential cryptanalysis of the ITUbee lightweight block cipher [**?**]. We evaluate the open-source solvers GLPK, HiGHS, and SCIP, as well as the commercial solvers Gurobi and CPLEX, using MILP models for 8, 10, and 12-round related-key differential attacks. As the number of rounds increases, the number of equations and constraints in the models also increases exponentially. Experiments were conducted on an 11th Gen Intel Core i7-1165G7 processor with 32 GB of RAM. Our results demonstrate that commercial solvers (Gurobi and CPLEX) significantly outperform open-source alternatives, achieving speedups of up to 94× compared to GLPK for the 12-round model. This study was prepared to guide those who will choose a solver for the MILP model. The comparison results obtained may differ even if the number of equations and constraints in the model are the same, because the equations will be different.

1 Introduction

Lightweight block ciphers are critical for resource-constrained environments such as embedded systems, IoT devices, and sensor networks. ITUbee is a software-oriented lightweight block cipher specifically designed for resource-constrained devices with limited computational power and memory [**?**]. The cipher features a Feistel structure consisting of 20 rounds, operating on an 80-bit block size with an 80-bit key. The encryption process incorporates a key whitening layer at both the beginning and end of the 20-round structure, employing substitution (F function) and linear transformation (L function) to provide security against various attacks.

Cryptanalytic attacks on block ciphers can be categorized into several fundamental types. Differential cryptanalysis, first introduced by Biham and Shamir [**?**], exploits statistical properties of plaintext-ciphertext pairs to

recover keys. Related-key differential cryptanalysis extends this paradigm by considering differences in both the plaintext and the encryption key, allowing attackers to exploit structural relationships between keys. This attack model is particularly relevant for cipher evaluation as it represents a more powerful threat than standard differential cryptanalysis.

In recent years, Mixed Integer Linear Programming (MILP) has emerged as a systematic and automated approach to finding optimal differential and related-key differential characteristics, significantly advancing the field of cryptanalysis [**?**]. The advantage of MILP-based cryptanalysis lies in its automation: researchers can construct objective functions and constraint systems that model the cipher's operations, and solvers can systematically search for optimal (minimum-weight) differential paths with proven guarantees of optimality.

However, the computational complexity of MILP-based related-key differential cryptanalysis depends critically on solver performance. Different MILP solvers employ different algorithms, heuristics, and optimization strategies, leading to substantial variations in solution time. This paper addresses a practical question of significant importance to cryptanalysts: which MILP solver should be selected for related-key differential cryptanalysis, and what performance improvements can be expected with different solver choices?

2 Methodology

2.1 ITUbee Cipher Design

TUbee [**?**] is a lightweight software-oriented block cipher specifically designed for resource-constrained devices which include a microcontroller and have a limited battery power such as sensor nodes in wireless sensor networks.

The cipher has Fiestel structure which consists of 20 rounds. İt has 80 bits block size and 80 bits key size. The round structure of the cipher is given in Figure **??**. Cipher also has a key whitening layer in the beginning and end of the 20 rounds. The encryption process of the cipher is given in Algorithm **??**. Round constants of the cipher is given in Table **??**

Figure 1: ITUbee round structure

[H] $X_1 \leftarrow P_L \oplus K_L \quad X_0 \leftarrow P_R \oplus K_R \quad i = 1$ to 20 $i \in \{1, 3, \ldots, 19\}$ $RK \leftarrow K_R \quad RK \leftarrow K_L \quad X_{i+1} \leftarrow X_{i-1} \oplus F(L(RK \oplus RC_i \oplus F(X_i)))$ Round constant is added to the rightmost 16 bits $C_L \leftarrow X_{20} \oplus K_R \;\; C_R \leftarrow X_{21} \oplus K_L$ Pseudo code for ITUbee encryption

2.1.1 L Function

The L funtion which has branch number 4 is computed as,
$L(a \parallel b \parallel c \parallel d \parallel e) = (e \oplus a \oplus b) \parallel (a \oplus b \oplus c) \parallel (b \oplus c \oplus d) \parallel (c \oplus d \oplus e) \parallel (d \oplus e \oplus a)$
This function can also be represented as matrix multiplication using rotational matrix:

$$\begin{pmatrix} 1 & 1 & 0 & 0 & 1 \\ 1 & 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 & 1 \\ 1 & 0 & 0 & 1 & 1 \end{pmatrix} \begin{pmatrix} a \\ b \\ c \\ d \\ e \end{pmatrix} = \begin{pmatrix} e \oplus a \oplus b \\ a \oplus b \oplus c \\ b \oplus c \oplus d \\ c \oplus d \oplus e \\ d \oplus e \oplus a \end{pmatrix}$$

2.1.2 F Function

The F function is computed as,

$$F(X) = S(L(S(X)))$$

S is the S-box used in AES [**?**]. S is the only non-linear structure in the ITUbee algorithm. 40 bit input of S is splitted into 8 bits then enters the s-box.

$$S(a \parallel b \parallel c \parallel d \parallel e) = s[a] \parallel s[b] \parallel s[c] \parallel s[d] \parallel s[e]$$

2.1.3 Key Schedule

ITUbee has no key schedule. The algorithm splits the 80 bit key into two 40 bit parts K_L and K_R and uses the right part of the key in odd rounds and the left part of the keys in even rounds.

2.2 Solver Specifications

This section provides detailed descriptions of the five MILP solvers evaluated in our comparative study.

2.2.1 GLPK (GNU Linear Programming Kit)

GLPK is a free and open-source linear programming solver developed as part of the GNU project [**?**]. Version 4.65 was used in our experiments. GLPK implements the revised simplex method for solving linear programming problems and employs branch-and-cut algorithms for handling integer constraints. While GLPK provides reliable and deterministic solutions, it lacks the advanced heuristics and parallel processing capabilities of commercial solvers, making it relatively slower for complex MILP instances. GLPK is widely used in academic research due to its accessibility and reasonable performance on small to medium-sized problems.

2.2.2 HiGHS (High-performance Interior-point Solver)

HiGHS (version 0.9) is a modern open-source linear and mixed-integer programming solver developed by the University of Edinburgh [**?**]. It implements advanced algorithms including the simplex method, interior-point method, and active-set method for LP problems, combined with parallel branch-and-bound techniques for MILP. HiGHS was designed with computational efficiency in mind, featuring parallel processing capabilities and modern data structures. Despite being open-source, HiGHS achieves significantly better performance than GLPK on many problem instances, particularly benefiting from multi-core processors through parallelization.

2.2.3 SCIP (Solving Constraint Integer Programs)

SCIP (version 7.0) is a powerful open-source constraint integer programming framework developed by the Zuse Institute Berlin [**?**]. Beyond basic MILP, SCIP supports advanced problem classes including mixed-integer nonlinear programming and quadratically constrained problems. SCIP incorporates sophisticated algorithmic techniques including cutting planes, primal heuristics, and symmetry breaking. Among open-source solvers, SCIP consistently demonstrates superior performance, often approaching the solution quality of commercial solvers for many problem classes. Its flexibility and strength make it a preferred choice for academic researchers with demanding computational requirements.

2.2.4 Gurobi Optimizer

Gurobi (version 10.0) is a leading commercial MILP solver developed by Gurobi Optimization [**?**]. It employs state-of-the-art algorithms including barrier methods (interior-point), parallel branch-and-cut, and advanced presolve techniques. Gurobi features sophisticated primal and dual heuristics for finding high-quality solutions quickly, combined with proven-effective cutting plane strategies. The solver automatically distributes computation across multiple processor cores, with dynamic load balancing. Gurobi's extensive tuning and parameter options allow fine-grained control over the solving process. It is widely deployed in industry and has become a benchmark standard for commercial solver performance.

2.2.5 CPLEX (IBM CPLEX Optimization Studio)

CPLEX (version 12.10) is IBM's flagship commercial optimization solver with decades of refinement [**?**]. It implements multiple algorithms including primal and dual simplex, barrier methods with crossover, and advanced branch-and-cut with dynamic strategy selection. CPLEX features parallel algorithms that can leverage multiple processor cores effectively. The solver

includes sophisticated preprocessing, probing techniques, and performance tuning capabilities. CPLEX has established itself as the de facto standard in enterprise optimization applications and represents the highest maturity level in commercial MILP solving.

2.2.6 Experimental Configuration

All solvers were executed with default parameters to ensure fair and reproducible comparison. This approach reflects typical usage patterns where practitioners rely on default settings without extensive parameter tuning. Each solver was allowed unlimited computation time (no time limits were imposed) to find optimal solutions. Experiments were conducted on an 11th Gen Intel Core i7-1165G7 processor operating at 2.80 GHz with 32 GB of system RAM, running Ubuntu Linux. This ensures all solvers had equal access to computational resources, with no artificial constraints or advantages.

3 Results and Analysis

Comprehensive benchmarking was conducted on MILP models for 8, 10, and 12-round related-key differential attacks on ITUbee. The results demonstrate clear performance distinctions between solver categories, with significant implications for practical related-key differential cryptanalysis.

3.1 8-Round Results

3.2 10-Round Results

3.3 12-Round Results

3.4 Discussion

The experimental results reveal several important trends:

1. **Commercial vs. Open-Source Gap**: Commercial solvers (Gurobi and CPLEX) consistently outperform open-source alternatives, with speedups increasing dramatically for larger problem sizes. For the 12-round model, Gurobi achieves a 94.6× speedup over GLPK.

2. **Problem Scaling**: As the problem complexity increases (8 to 12 rounds), the performance gap between solvers widens. This suggests that more sophisticated optimization algorithms in commercial solvers are particularly effective on harder instances.

3. **Open-Source Alternatives**: Among open-source solvers, SCIP consistently outperforms both GLPK and HiGHS, achieving 2.9× speedup on 8-rounds and 12× speedup on 12-rounds compared to GLPK.

4. **Commercial Solver Parity**: Gurobi and CPLEX show comparable performance, with Gurobi slightly faster on 10 and 12-round models, while CPLEX excels on the 8-round problem.

The timing analysis demonstrates that solver choice is critical for practical cryptanalysis. For resource-constrained deployments, SCIP provides a reasonable alternative to commercial solvers. However, for optimal performance, commercial solutions are strongly recommended.

4 Conclusion

This paper provides a systematic empirical comparison of five state-of-the-art MILP solvers in the context of cryptanalytic applications, specifically related-key differential attacks on the ITUbee lightweight block cipher. Our findings clearly demonstrate that solver selection significantly impacts the practical feasibility of MILP-based related-key differential cryptanalysis.

Key conclusions include:

1. Commercial MILP solvers (Gurobi and CPLEX) are substantially more efficient than open-source alternatives, justifying their use in computationally demanding related-key differential cryptanalytic applications.

2. For organizations without access to commercial solvers, SCIP represents a robust open-source option, providing reasonable performance for moderate problem sizes in related-key differential attacks.

3. The performance advantage of commercial solvers increases with problem complexity, suggesting that their advanced algorithms are particularly effective for hard instances of related-key differential attacks.

4. Cryptanalysts should carefully evaluate solver options based on their specific constraints (computational budget, licensing availability, problem size) when conducting related-key differential cryptanalysis.

Future work could extend this analysis to other cipher families and attack models, evaluate hybrid solver strategies, and investigate parallel solving approaches for further performance gains in MILP-based cryptanalysis.

References

[1] Kaplan, H. İ. (2026). "Usage of Mixed Integer Linear Programming in Cryptanalysis of Block Ciphers." *Cryptanalysis Laboratory, TÜBİTAK BİLGEM*, 2026.

[2] Mouha, N., Wang, Q., Gu, D., & Preneel, B. (2012). "Differential and Linear Cryptanalysis of Reduced-Round Simon." *IACR Transactions on Symmetric Cryptology*, 1, 79–104.

[3] Biham, E., & Shamir, A. (1990). "Differential Cryptanalysis of DES-like Cryptosystems." *Journal of Cryptology*, 4(1), 3–72.

[4] Free Software Foundation. (2024). "GNU Linear Programming Kit (GLPK)." Retrieved from https://www.gnu.org/software/glpk/

[5] Hall, J. A. J., Lasdon, L. S., Steiger, S. M., et al. (2023). "HiGHS: An Open Source Solver for Mixed-Integer Programming." *Technical Report*, University of Edinburgh.

[6] Gleixner, A. M., Bastubbe, M., Bestuzheva, K., et al. (2023). "The SCIP Optimization Suite 8.0." *Technical Report*, Zuse Institute Berlin.

[7] Gurobi Optimization, LLC. (2024). "Gurobi Optimizer Reference Manual." Retrieved from https://www.gurobi.com/documentation/

[8] IBM. (2024). "CPLEX Optimizer Documentation." Retrieved from https://www.ibm.com/docs/en/icos/

Round	**Round Constant (Hex)**	**Round**	**Round Constant (Hex)**
1	0x1428	11	0x0a1e
2	0x1327	12	0x091d
3	0x1226	13	0x081c
4	0x1125	14	0x071b
5	0x1024	15	0x061a
6	0x0f23	16	0x0519
7	0x0e22	17	0x0418
8	0x0d21	18	0x0317
9	0x0c20	19	0x0216
10	0x0b1f	20	0x0115

Table 1: Round Constants of ITUbee algorithm

Table 2: Solution times for the 8-round model (687 variables, 2147 constraints)

Solver	Time (s)	Speedup
GLPK	257	1.0×
HiGHS	231	1.1×
SCIP	86	2.9×
Gurobi	37	6.9×
CPLEX	16	16.0×

Table 3: Solution times for the 10-round model (945 variables, 3182 constraints)

Solver	Time (s)	Speedup
GLPK	1610	1.0×
HiGHS	321	5.0×
SCIP	202	7.9×
Gurobi	34	47.4×
CPLEX	40	40.3×

Table 4: Solution times for the 12-round model (1203 variables, 4217 constraints)

Solver	Time (s)	Speedup
GLPK	4543	1.0×
HiGHS	1780	2.5×
SCIP	376	12.0×
Gurobi	48	94.6×
CPLEX	53	85.7×